KNAV · CVIČENÍ 2

Jak skládat objekty

Adapter · Bridge · Composite · Decorator · Facade · Flyweight

Šest vzorů. U každého jedna otázka, jeden příklad a kousek kódu.

PŘEHLED

Co přesně se snažíme zjednodušit?

Vzor	Hlavní otázka
Adapter	Jak spojit rozhraní, která k sobě nepasují?
Bridge	Jak oddělit dvě nezávislé osy variant?
Composite	Jak zacházet se stromem objektů jako s jedním?
Decorator	Jak za běhu přidat chování bez dědičnosti?
Facade	Jak zjednodušit práci se složitým podsystémem?
Flyweight	Jak ušetřit paměť u mnoha podobných objektů?

ADAPTER

„Přelož jedno rozhraní na druhé“

POUŽIJ, KDYŽ

máš hotovou třídu, jejíž rozhraní nepasuje k tomu, co klient očekává, a nechceš ji přepisovat.

PŘÍKLAD

starý zdroj vrací XML, ale aplikace pracuje s JSON. Adaptér XML převede, klient o tom neví.

```
interface JsonDataService { String getJsonData(); }

class XmlToJsonAdapter implements JsonDataService {
  private final XmlDataService adaptee;

  public String getJsonData() {
    return toJson(adaptee.getXmlData());
  }
}
```

JEDNOU VĚTOU Adapter převádí rozhraní existující třídy na takové, které klient očekává.

BRIDGE

„Odděl abstrakci od implementace“

POUŽIJ, KDYŽ

máš dvě nezávislé osy variant a nechceš násobit počet tříd (každá kombinace jako vlastní podtřída).

PŘÍKLAD

ovladač (základní, pokročilý) a zařízení (televize, rádio). Ovladač drží odkaz na zařízení a deleguje na něj práci. Každý ovladač tak funguje s každým zařízením.

```
interface Device { void setVolume(int level); }   // implementace
class TV implements Device { ... }
class Radio implements Device { ... }
class RemoteControl {                 // abstrakce
  protected final Device device;      // „most“
  RemoteControl(Device d) { device = d; }
  void volumeUp() { device.setVolume(+1); }
}
class AdvancedRemote extends RemoteControl {
  void mute() { device.setVolume(0); }
}
new AdvancedRemote(new TV()).mute();      // ovládá televizi
new AdvancedRemote(new Radio()).mute();   // totéž s rádiem
```

JEDNOU VĚTOU Bridge odděluje abstrakci od implementace, aby se mohly měnit nezávisle.

COMPOSITE

„Se skupinou zacházej jako s jedním objektem“

POUŽIJ, KDYŽ

data tvoří stromovou strukturu a klient má s jednotlivcem i s celou větví zacházet stejně.

PŘÍKLAD

firma se odděleními a zaměstnanci. Plat zaměstnance i celého oddělení se získá stejným voláním getSalary().

```
interface EmployeeComponent {
  double getSalary();
}
class Employee implements EmployeeComponent { ... }
class Department implements EmployeeComponent {
  private final List<EmployeeComponent> members;
  public double getSalary() {
    return members.stream()
        .mapToDouble(EmployeeComponent::getSalary)
        .sum();
  }
}
```

JEDNOU VĚTOU Composite umožňuje zacházet s jednotlivými objekty i se skupinami stejně.

DECORATOR

„Přidej chování obalením“

POUŽIJ, KDYŽ

chceš objektu za běhu přidat vlastnosti a nechceš kvůli každé kombinaci dědit novou třídu.

PŘÍKLAD

zpráva se může šifrovat, komprimovat nebo počítat slova. Dekorátory se dají libovolně skládat do sebe.

POZOR

záleží na pořadí obalení. Šifrování před kompresí dá jiný výsledek než naopak.

```
Message message = new SimpleMessage("Hello World");
Message composed =
    new WordCountDecorator(
        new CompressionDecorator(
            new EncryptionDecorator(message)));

composed.getContent();
```

JEDNOU VĚTOU Decorator přidává objektu chování tím, že ho obalí objektem stejného rozhraní.

FACADE

„Jedno jednoduché rozhraní k složitému systému“

POUŽIJ, KDYŽ

klient nemá znát a ručně ovládat desítky podsystémů, stačí mu pár jednoduchých operací.

PŘÍKLAD

domácí kino. Metoda watchMovie() zhasne světla, zapne projektor, zvuk i přehrávač ve správném pořadí.

```
class HomeTheaterFacade {
  void watchMovie(String movie) {
    light.dim();
    projector.on();
    sound.setVolume(5);
    dvd.play(movie);
  }
}

theater.watchMovie("The Matrix");
```

JEDNOU VĚTOU Facade poskytuje jednoduché rozhraní k souboru složitějších tříd.

FLYWEIGHT

„Sdílej společná data mezi mnoha objekty“

POUŽIJ, KDYŽ

potřebuješ obrovské množství podobných objektů a paměť je omezená.

PŘÍKLAD

les s tisíci stromů. Druh stromu (název, barva, textura) se sdílí, každý strom si pamatuje jen pozici.

POZOR

sdílená data musí být neměnná, jinak se změna projeví u všech.

```
record TreeType(String name, String color, String texture) {}
record Tree(int x, int y, TreeType type) {}

static TreeType getTreeType(String n, String c, String t) {
  return TYPES.computeIfAbsent(n + "|" + c + "|" + t,
      key → new TreeType(n, c, t));
}
```

JEDNOU VĚTOU Flyweight šetří paměť sdílením společných dat mezi velkým množstvím objektů.

SHRNUTÍ

Pro rychlé zapamatování

Vzor	Jednou větou
Adapter	Adapter převádí rozhraní existující třídy na takové, které klient očekává.
Bridge	Bridge odděluje abstrakci od implementace, aby se mohly měnit nezávisle.
Composite	Composite umožňuje zacházet s jednotlivými objekty i se skupinami stejně.
Decorator	Decorator přidává objektu chování tím, že ho obalí objektem stejného rozhraní.
Facade	Facade poskytuje jednoduché rozhraní k souboru složitějších tříd.
Flyweight	Flyweight šetří paměť sdílením společných dat mezi velkým množstvím objektů.

A TEĎ DO KÓDU

Podíváme se, jak to vypadá doopravdy

Spusťte Exercise2PoC a podívejte se na výstup.

Pak otevřeme balíček každého vzoru a projdeme ho společně.

Zkuste něco změnit: další variantu, jinou hodnotu, jiné nastavení.

github.com/JakubObornik/KNAVProject

```
src/exercises/exercise2/
├─ Exercise2PoC.java
└─ patterns/
   ├─ adapter/
   ├─ bridge/
   ├─ composite/
   ├─ decorator/
   ├─ facade/
   └─ flyweight/
```



Odkaz na projekt